

Experiment 2 Report: Evaluation Function Comparison

1. Objective

Experiment 2 compares three evaluation functions under the same search framework.

Unlike Experiment 1, this experiment does not compare Minimax and Alpha-Beta.

The search algorithm is fixed as:

Alpha-Beta + move ordering

The goal is to observe how different evaluation ideas affect:

- best move selection;
- best score;
- explored nodes;
- cutoffs;
- runtime.

2. Compared Evaluations

Eval A: Consecutive Length

Eval A only considers consecutive segment length. It tends to favor moves that immediately extend an existing line.

Eval B: Open / Blocked Shape

Eval B considers both segment length and whether the segment has open ends. It can distinguish open shapes from blocked shapes, such as open threes and blocked threes.

Eval C: Five-Cell Window Potential

Eval C scans all length-5 windows and scores potential five-in-a-row spaces. It also includes a small center bonus. This can make it more sensitive to long-term space and multi-directional potential, but it is computationally more expensive.

3. Controlled Conditions

All evaluations use the same:

- board size: 15 x 15 ;
- search algorithm: Alpha-Beta + move ordering ;
- candidate generator: generate_candidate_moves(board, radius=2) ;
- depths: 1, 2, 3 ;
- fixed test positions: positions.py .

Depth 4 is not tested in this experiment.

The fixed boards are saved in terminal format:

- results/evaluation_positions.txt

4. Fixed Positions

The experiment uses five purpose-built positions:

- E1_length_pressure : checks whether length-only scoring prefers extending a longer local chain.
- E2_open_vs_blocked : compares blocked longer shapes against open shapes.
- E3_window_cross : gives Eval C a chance to value multi-window potential.
- E4_attack_defense : compares attacking moves against reducing opponent potential.
- E5_complex_midgame : compact midgame with several competing patterns.

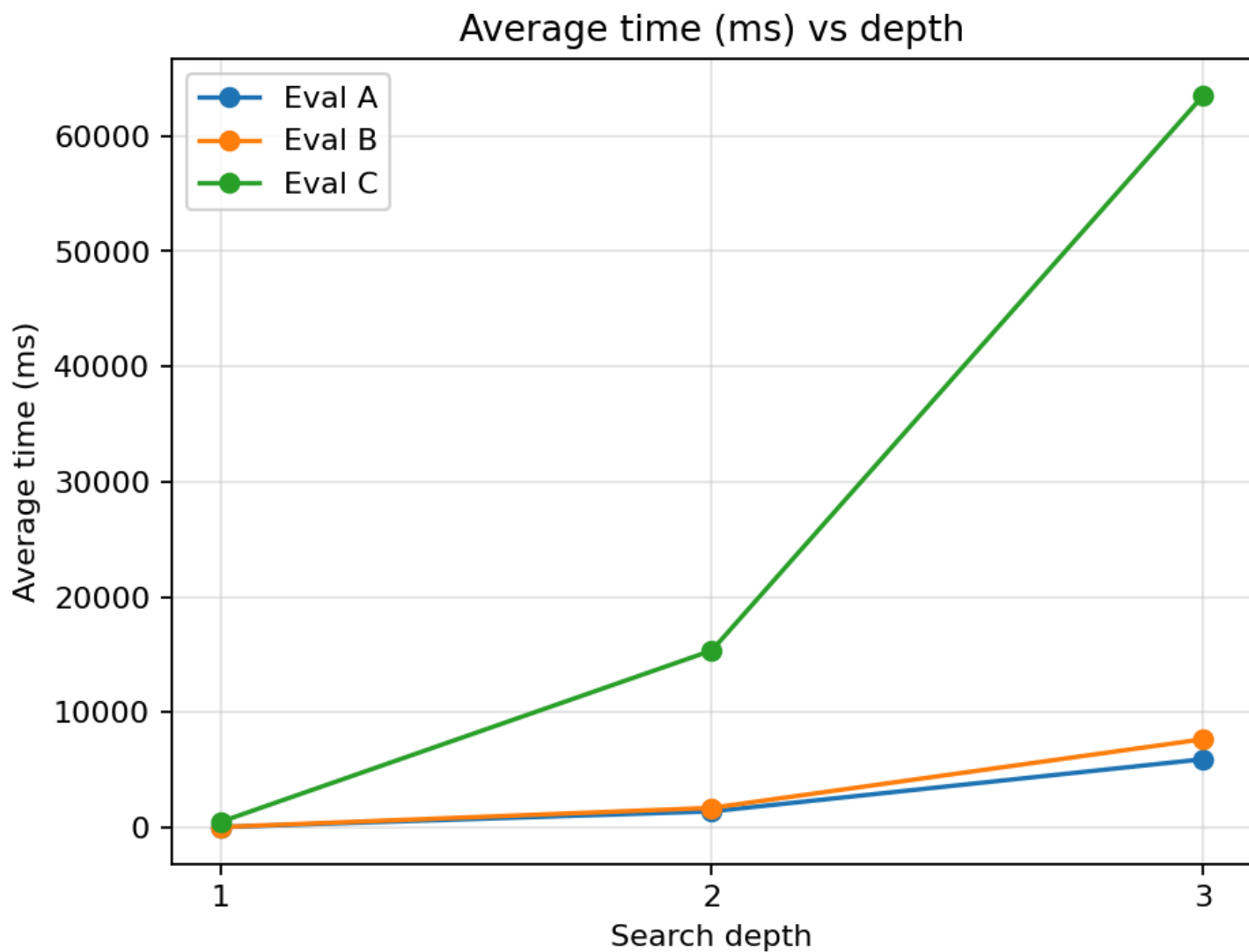
These positions are not random. Each one is designed to make evaluation behavior easier to explain.

5. Summary Table

depth	evaluation	positions	avg_time_ms	avg_nodes	avg_cutoffs	avg_candidate_count
1	Eval A	5	42.7976	54.6	0.0	53.6
1	Eval B	5	48.0968	54.6	0.0	53.6
1	Eval C	5	492.8529	54.6	0.0	53.6
2	Eval A	5	1404.0459	162.8	52.6	53.6
2	Eval B	5	1712.4998	194.2	52.0	53.6
2	Eval C	5	15368.7781	206.8	51.8	53.6
3	Eval A	5	5932.5626	3593.6	107.2	53.6

depth	evaluation	positions	avg_time_ms	avg_nodes	avg_cutoffs	avg_candidate_count
3	Eval B	5	7665.048	3590.6	117.6	53.6
3	Eval C	5	63505.0764	3607.2	118.6	53.6

6. Runtime Comparison



Eval A is the fastest evaluation. Eval B is slightly slower because it also checks open and blocked ends. Eval C is much slower because it scans all length-5 windows during evaluation.

At depth 3:

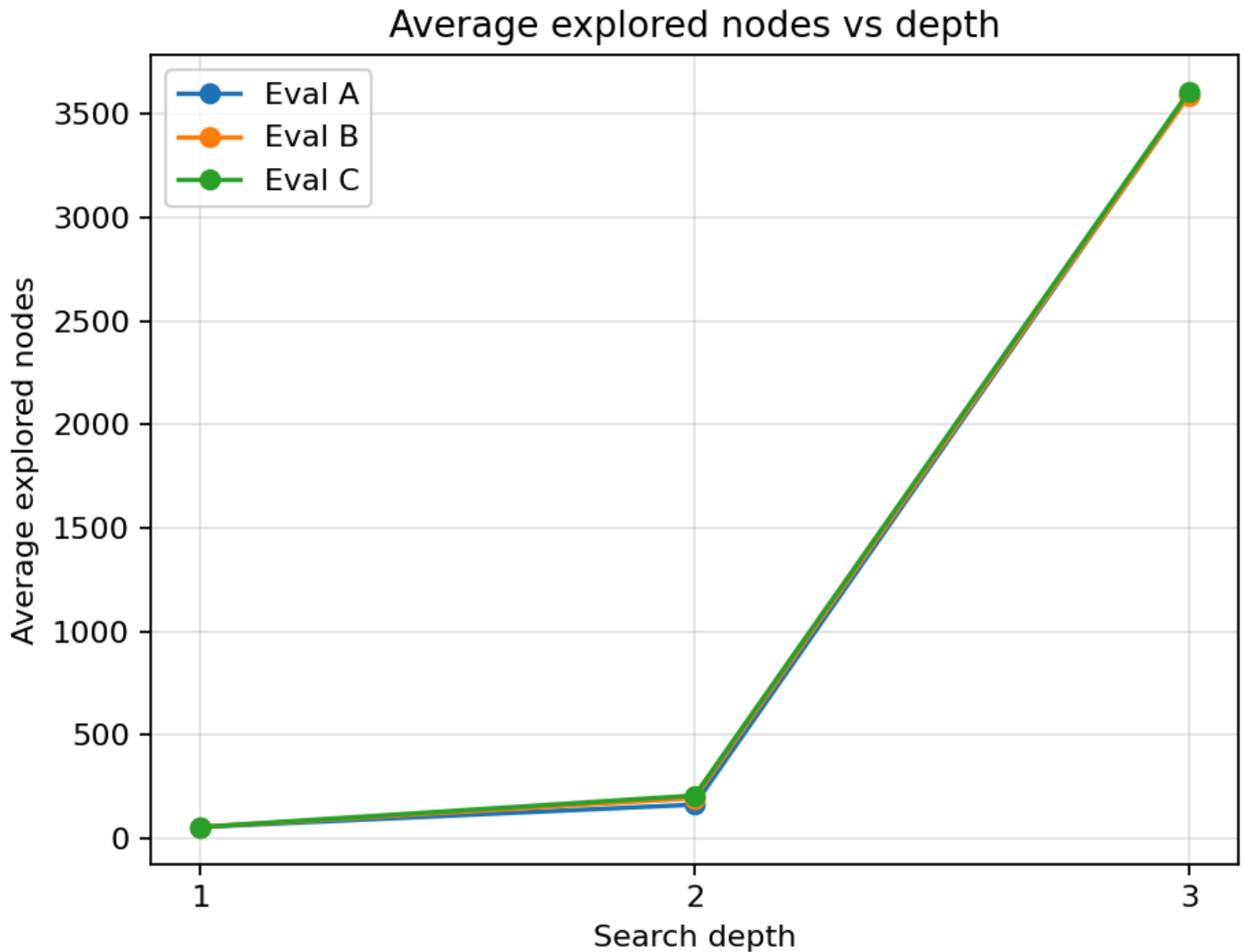
```

Eval A avg time: 5932.5626 ms
Eval B avg time: 7665.0480 ms
Eval C avg time: 63505.0764 ms

```

This confirms that Eval C has a significantly higher computational cost.

7. Node Comparison



The average node counts are similar across all three evaluations because the search algorithm and candidate generator are fixed. However, evaluation still affects move ordering and alpha/beta boundary updates, so the exact node counts are not identical.

At depth 3:

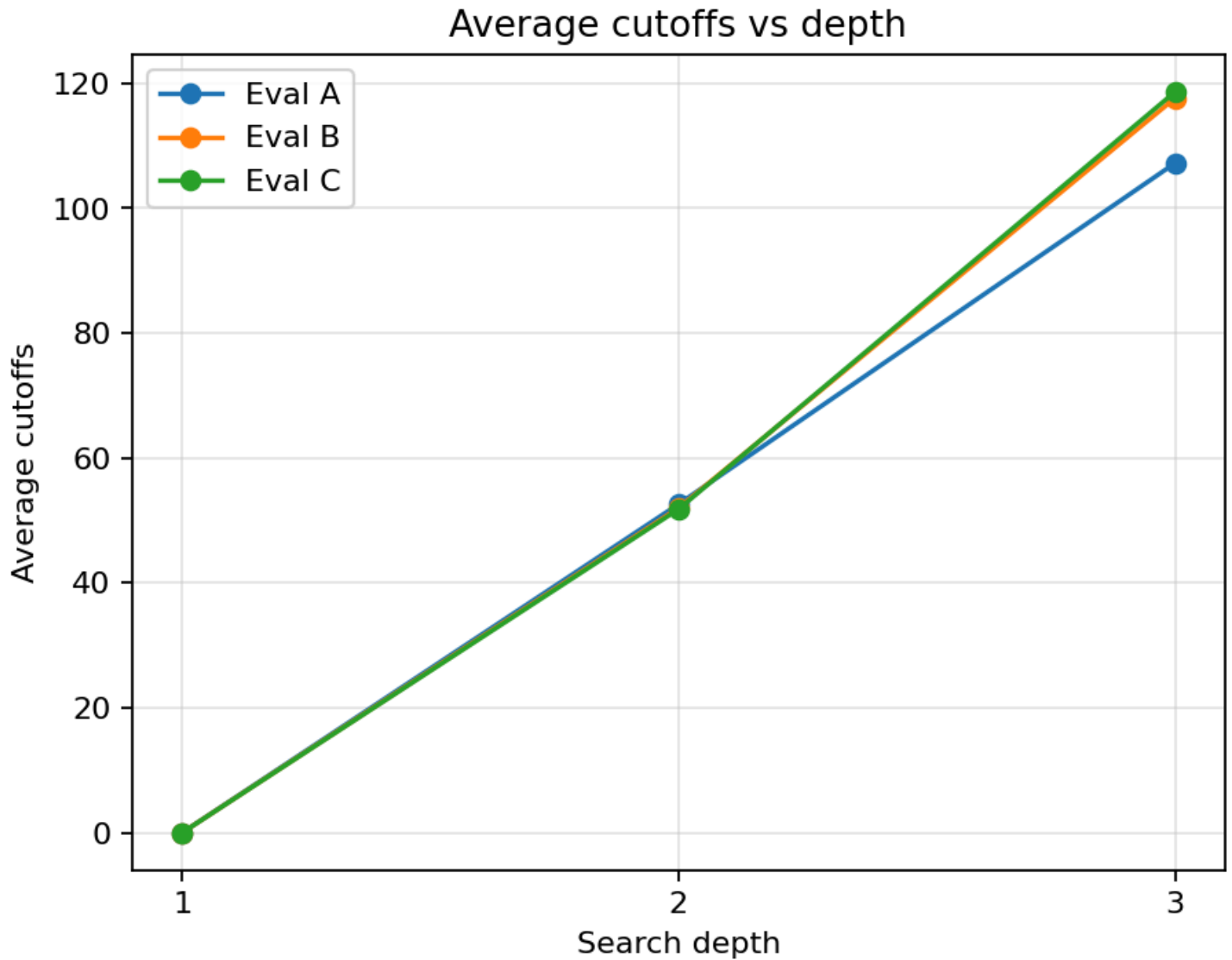
Eval A avg nodes: 3593.6

Eval B avg nodes: 3590.6

Eval C avg nodes: 3607.2

The main cost difference is therefore not node count, but the cost of evaluating each node.

8. Cutoffs



Cutoffs increase as depth grows. Because move ordering depends on the evaluation function, different evaluations can produce slightly different cutoff behavior.

At depth 3:

Eval A avg cutoffs: 107.2

Eval B avg cutoffs: 117.6

Eval C avg cutoffs: 118.6

Eval B and Eval C produced slightly more cutoffs at depth 3, which suggests that their move ordering produced somewhat better alpha/beta boundaries in these positions.

9. Best Move Differences

The best move difference table is saved in:

- `results/evaluation_best_move_diff.csv`

Out of 15 position-depth cases, the three evaluations selected different moves in 7 cases.

Important examples:

position	depth	Eval A move	Eval B move	Eval C move	observation
E1_length_pressure	2	7 9	8 8	8 8	Eval A extends a local line; Eval B/C prefer another shape.
E2_open_vs_blocked	2	8 5	8 9	8 9	Eval B/C choose a different open-shape move.
E3_window_cross	2	5 9	5 9	9 5	Eval C chooses a different window-potential move.
E4_attack_defense	1	6 10	6 6	6 6	Eval A attacks differently; Eval B/C prefer another tactical point.

This confirms that the evaluations are not merely different weights producing the same behavior. They can lead to different decisions under the same search algorithm and same candidate set.

10. Interpretation

The results support the expected behavior:

- Eval A is fastest and tends to favor direct line extension.
- Eval B is still relatively efficient and better represents open/blocked shapes.
- Eval C is much more expensive because it evaluates all five-cell windows.
- Eval C can select different moves when window potential differs from local length or openness.
- More complex evaluation does not automatically mean better shallow-depth decisions. With depth limited to 1, 2, and 3, Eval C's extra cost is clear, while its decision advantage depends on the position.

11. How to Reproduce

Run the benchmark:

```
python experiments/experiment2_evaluations/compare_evaluations.py
```

Generate plots:

```
python experiments/experiment2_evaluations/plot_evaluation_comparison.py
```

Export the fixed positions:

```
python experiments/experiment2_evaluations/export_positions.py
```

Install plotting dependency if needed:

```
python -m pip install -r requirements.txt
```